

7. Implement Johnson Trotter algorithm to generate permutation.

```
#include <stdio.h>

#define LEFT -1
#define RIGHT 1

void printPermutation (int perm[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d", perm[i]);
    printf("\n");
}

int getMobile (int perm[], int dir[], int n) {
    int mobile = 0;
    int mobile_index = -1;

    for (int i = 0; i < n; i++) {
        if (dir[perm[i]-1] == LEFT && i != 0) {
            if (perm[i] < perm[i-1] && perm[i] > mobile) {
                mobile = perm[i];
                mobile_index = i;
            }
        }
    }

    return mobile_index;
}

void johnsonTrotter (int n) {
    int perm[n], dir[n];

    for (int i = 0; i < n; i++)
```

```

perm[i] = i+1;
dir[i] = LEFT;
}
print Permutation (perm, n);
while (1) {
    int mobile_index = getMobile (perm, dir, n);
    if (mobile_index == -1)
        break;
    int mobile = perm[mobile_index];
    if (dir[mobile-1] == LEFT) {
        int temp = perm[mobile_index];
        perm[mobile_index] = perm[mobile_index-1];
        perm[mobile_index-1] = temp;
        mobile_index--;
    }
    else {
        int temp = perm[mobile_index];
        perm[mobile_index] = perm[mobile_index+1];
        perm[mobile_index+1] = temp;
        mobile_index++;
    }
}
for (int i=0; i<n; i++) {
    if (perm[i] > mobile) {
        dir[perm[i]-1] *= -1;
    }
}
print Permutation (perm, n);
}
}

```



```

int main() {
    int n;

    printf("Enter number of elements:");
    scanf("%d", &n);

    johnsonTrotter(n);

    return 0;
}

```

output:

Enter number of elements: 3

1 2 3

1 3 2

3 1 2

3 2 1

2 3 1

2 1 3